



Commits Convencionales

Especificación para dar significado a los mensajes de los commits haciéndolos legibles para máquinas y humanos

Lee la especificación

GitHub



Commits Convencionales 1.0.0

Resumen

La especificación de Commits Convencionales es una convención ligera sobre los mensajes de commits. Proporciona un conjunto sencillo de reglas para crear un historial de commits explícito; lo que hace más fácil escribir herramientas automatizadas encima del historial. Esta convención encaja con **SemVer**, al describir en los mensajes de los commits las funcionalidades, arreglos, y cambios de ruptura hechos.

El mensaje del commit debe ser estructurado de la siguiente manera:

```
<tipo>[ámbito opcional]: <descripción>

[cuerpo opcional]

[nota(s) al pie opcional(es)]
```

El commit contiene los siguientes elementos estructurales, para comunicar la intención a los consumidores de tu librería:

1. **fix**: un commit de *tipo* `fix` corrige un error en la base del código (se correlaciona con `PATCH` en el Versionado Semántico).
2. **feat**: un commit de *tipo* `feat` introduce una nueva funcionalidad en la base del código (se correlaciona con `MINOR` en el Versionado Semántico).
3. **BREAKING CHANGE**: un commit que contiene la nota al pie `BREAKING CHANGE:` , o que agrega un `!` después del tipo/ ámbito, introduce un cambio de ruptura de API (se correlaciona con `MAJOR` en el Versionado Semántico). Un BREAKING CHANGE puede ser parte de commits de cualquier *tipo*.
4. *tipos* distintos a `fix`: y `feat`: están permitidos, por ejemplo `@commitlint/config-conventional` (basados en la **convención de Angular**) que recomienda `build:` , `chore:` , `ci:` , `docs:` , `style:` , `refactor:` , `perf:` , `test:` , y otros.
5. *notas al pie* distintas de `BREAKING CHANGE: <descripción>` pueden ser añadidas y siguen una convención similar al **formato git trailer**.

Tipos adicionales no son obligatorios en la especificación de Commits Convencionales, y no tienen un efecto implícito en el Versionado Semántico (al menos que incluyan un BREAKING CHANGE). Un ámbito puede ser añadido al tipo de un commit, para proveer información adicional contextual y debe ser contenido entre paréntesis, ej., `feat(parser): add ability to parse arrays`.

Ejemplos

Mensaje de commit con descripción y cambio de ruptura en la nota al pie

```
feat: allow provided config object to extend other configs

BREAKING CHANGE: `extends` key in config file is now used for extending other config files
```

Mensaje de commit con `!` para llamar la atención al cambio de ruptura

```
refactor!: drop support for Node 6
```

Mensaje de commit con ambos `!` y BREAKING CHANGE en la nota al pie

```
refactor!: drop support for Node 6

BREAKING CHANGE: refactor to use JavaScript features not available in Node 6.
```

Mensaje de commit sin cuerpo

```
docs: correct spelling of CHANGELOG
```

Mensaje de commit con ámbito

```
feat(lang): added polish language
```

Mensaje de commit con cuerpo multi-párrafo y múltiples notas al pie

```
fix: correct minor typos in code

see the issue for details

on typos fixed.

Reviewed-by: Z
Refs #133
```

Especificación

Las palabras clave "DEBE" ("MUST"), "NO DEBE" ("MUST NOT"), "REQUERIDO" ("REQUIRED"), "DEBERÁ" ("SHALL"), "NO DEBERÁ" ("SHALL NOT"), "DEBERÍA" ("SHOULD"), "NO DEBERÍA" ("SHOULD NOT"), "RECOMIENDA" ("RECOMMENDS"), "PUEDE" ("MAY") y "OPCIONAL" ("OPTIONAL") en este documento se deben interpretar como se describe en **RFC 2119**.

1. Los commits DEBEN iniciarse con un prefijo de tipo que consiste en un sustantivo, `feat` , `fix` , etc., seguido del ámbito OPCIONAL, `!` OPCIONAL, y dos puntos y un espacio REQUERIDO.
2. El tipo `feat` DEBE ser usado cuando un commit agrega una nueva funcionalidad a la aplicación o librería.
3. El tipo `fix` DEBE ser usado cuando el commit representa una corrección a un error en el código de la aplicación (bug).
4. Un ámbito PUEDE ser añadido después de un tipo. Un ámbito DEBE consistir en un sustantivo que describa una sección de la base del código encerrado entre paréntesis, ej., `fix(parser):` .
5. Una descripción DEBE ir inmediatamente después de los dos puntos y el espacio del prefijo de tipo/ámbito. La descripción es resumen corto de los cambios realizados en el código, ej., *fix: array parsing issue when multiple spaces were contained in string.*.
6. Un cuerpo de commit más extenso PUEDE agregarse después de la descripción corta, dando información contextual adicional acerca de los cambios en el código. El cuerpo DEBE iniciar después de una línea en blanco después de la descripción.
7. Un cuerpo de commit es de forma-libre y PUEDE consistir de cualquier número de párrafos separados por una nueva línea.
8. Una o más notas al pie PUEDEN ser añadidas una línea en blanco después del cuerpo. Cada nota al pie DEBE consistir de una palabra clave, seguida ya sea por un separador `:<espacio> o <espacio>#` , seguido por un valor cadena (string) (esto está inspirado por la **convención git trailer**).
9. Una palabra clave de una nota al pie DEBE usar `-` en lugar de caracteres de espacios en blanco, ej., `Acked-by` (esto ayuda a diferenciar la sección de la nota al pie de un cuerpo multi párrafo). Se hace una excepción para `BREAKING CHANGE` , que también PUEDE ser usada como palabra clave.
10. Una nota al pie PUEDE contener espacios y líneas en blanco, y el parseo DEBE terminar cuando se observe el siguiente separador/clave.
11. Los cambios de ruptura DEBEN ser indicados en el prefijo de tipo/ámbito de un commit, o como una entrada en la nota al pie.
12. Si se incluye como una nota al pie, un cambio de ruptura DEBE consistir del texto en mayúsculas BREAKING CHANGE, seguido de dos puntos, y una descripción, ej., *BREAKING CHANGE: environment variables now take precedence over config files*.
13. Si se incluye en el prefijo de tipo/ámbito, cambios de ruptura DEBEN ser indicados por un `!` inmediatamente después de `:` . Si `!` es usado, `BREAKING CHANGE:` PUEDE ser omitido de la sección de la nota al pie, y la descripción del commit DEBERÁ ser usada para describir el cambio de ruptura.
14. Tipos diferentes a `feat` y `fix` PUEDEN ser usados en los mensajes de commit, ej., *docs: updated ref docs.*.
15. Las unidades de información que componen Commits Convencionales NO DEBEN ser tratados como implementadores sensitivos de caso, con la excepción de BREAKING CHANGE que DEBE ir en mayúsculas.
16. BREAKING-CHANGE DEBE ser sinónimo de BREAKING CHANGE, cuando se usa en una nota al pie.

¿Por qué usar Commits Convencionales?

- Generación automática de CHANGELOGs.
- Determinación automática de un salto de versión semántico (basado en los tipos de commits utilizados).
- Comunicar la naturaleza de los cambios a los demás integrantes del equipo, el público o cualquier otro interesado.
- Activar procesos de procesos y publicación.
- Hacer más fácil a otras personas contribuir al proyecto al permitirles explorar una historia de los commits más estructurada.

FAQ (Preguntas Frecuentes)

¿Cómo puedo trabajar con los mensajes de los commits en la etapa inicial de desarrollo?

Recomendamos trabajar como si ya hubieras lanzado tu producto. Típicamente *alguien*, incluso si son tus compañeros desarrolladores, están usando tu software. Ellos querrán saber qué se ha arreglado, qué se ha dañado, etc.

¿Los tipos en los títulos de los commits están en mayúsculas o en minúsculas?

Se puede utilizar cualquiera, pero es mejor ser coherente.

¿Qué hago si el commit se ajusta a más de uno de los tipos de commit?

Regresa y haz múltiples commits de ser posible. Parte de los beneficios de los Commits Convencionales es la habilidad para hacer commits y PRs más organizados.

¿No desalienta esto el desarrollo y la iteración rápida?

Desalienta moverse rápido de una forma desorganizada. Te ayuda a ser capaz de moverte rápido a largo plazo a través de proyectos con una gran variedad de contribuidores.

¿Pueden los Commits Convencionales llevar a los desarrolladores a limitar el tipo de commits que hacen ya que estarán pensando en los tipos previstos?

Los Commits Convencionales nos animan a hacer más de ciertos tipos de commits como *fixes*. Adicionalmente, la flexibilidad de los Commits Convencionales le permite a tu equipo generar sus propios tipos y cambiarlos a lo largo del tiempo.

¿Cómo se relaciona esto con SemVer?

El tipo de commit `fix` se traduce a un cambio de versión `PATCH` . El tipo de commit `feat` se traduce a un cambio de versión `MINOR` . Commits con el texto `BREAKING CHANGE` , sin importar su tipo, se traducen a un cambio de versión `MAJOR` .

¿Cómo debo versionar mis extensiones a la especificación de Commits Convencionales, e.g. @jameswomack/conventional-commit-spec ?

Recomendamos usar SemVer para liberar tu propia extensión a esta especificación (¡Y te animamos a hacer esta extensión!)

¿Qué debo hacer si por accidente uso un tipo de commit equivocado?

Cuando utilizaste un tipo que es de la especificación pero del tipo correcto, e.g. `fix` en lugar de `feat`

Antes de combinar o liberar el error, recomendamos usar `git rebase -i` para editar el historial de los commits. Después de que se ha liberado, la limpieza será distinta de acuerdo a las herramientas y procesos que hayas utilizado.

Cuando usas un tipo que no está en la especificación, ej. `feet` en lugar de `feat`

En el peor de los escenarios, no es el fin del mundo si aparece un commit que no cumple con las especificaciones de los Commits Convencionales. Simplemente significa que el commit será ignorado por las herramientas que se basen en esta especificación.

¿Deben todos los que contribuyen a mi proyecto usar esta especificación?

¡No! Si usas un flujo de trabajo basado en `squash` los líderes del proyecto pueden limpiar el mensaje en el momento en que se incorpora, sin agregar cargas adicionales a quienes contribuyen casualmente. Un flujo de trabajo común para esto es configurar tu sistema de git para que haga el `squash` de manera automática de un pull request y presente al líder del proyecto un formulario para que ingrese el mensaje de commit correcto al momento de hacer el merge.

¿Cómo maneja Commits Convencionales el revertir código?

Revertir código puede ser complicado: ¿Estás revirtiendo múltiples commits? ¿Si reviertes una funcionalidad, debería ser la siguiente versión un *patch*?

Commits de herramientas no hace un esfuerzo explícito para definir el comportamiento al revertir. En lugar de eso le dejamos a los autores de las herramientas la flexibilidad de los *tipos* y *notas al pie* para desarrollar la lógica para manejar los *reverts*.

Una recomendación es usar el tipo `revert` , y una nota al pie que haga referencia a los commit SHAs que son revertidos:

```
revert: let us never again speak of the noodle incident

Refs: 676104e, a215868
```

